Session 24 Homework — Solution Walkthrough

Status: PREPARED DRAFT. Published with the official solution, for independent study.

What the homework asked you to prove

A real combined multi-criteria filter (the real backend's own `status` + `title` query parameters, sent together) plus a real client-derived sort over whatever the backend already filtered — then a real, reproduced written answer on the Tailwind v4/v3 version pin, and a real fix for TEAMWORK's own disclosed theme-persistence gap.

What the official solution changes

CORE: two new real Client Components — `TitleSearch.tsx` (the identical real URL-driven pattern as `StatusFilter`, for free text) and `SortableIssueList.tsx` (a real, client-owned sort over the already backend-filtered `issues` prop) — plus Project Detail's own fetch extended to send both `status` and `title` together, and its own render wired to use both new components.

STRETCH: no code — a real, reproduced written answer.

CHALLENGE: `web/src/context/ThemeContext.tsx` — both real `useEffect` calls edited so a stored theme choice survives a reload.

Files changed

- `web/src/components/issues/TitleSearch.tsx` (new)
- `web/src/components/issues/SortableIssueList.tsx` (new)
- `web/src/app/workspaces/[id]/projects/[projectId]/page.tsx` (CORE: fetch + render)
- `web/src/context/ThemeContext.tsx` (CHALLENGE: both `useEffect` calls)

Step-by-step implementation

1. **`TitleSearch` skeleton:** reads the real `title` URL parameter, renders a plain uncontrolled `<input defaultValue={...}>` with no `onChange` yet — the identical real skeleton-first discipline `StatusFilter` already used.
2. **`TitleSearch` write-back:** an `onChange` handler pushes the real typed value into the URL's own `title` search parameter, the identical real `URLSearchParams` pattern as `StatusFilter`.
3. **`SortableIssueList` skeleton:** takes the already-filtered `issues` prop, holds a real `sortBy` piece of CLIENT-owned state (`useState`), and computes a real `sorted` array — this is genuinely new: backend-owned filter state (the URL) and client-owned sort state coexist, neither one faked as the other, since the real backend has no sort parameter at all.
4. **`SortableIssueList` sort control:** a real, CONTROLLED `<select value={sortBy}>` — unlike `TitleSearch`'s uncontrolled input, this component must re-render its own sorted list the instant the selection changes, which requires React to own the value.

5. **`SortableIssueList` render:** maps over the real `sorted` array (never the raw `issues` prop) with the identical real `Card` / `StatusBadge` / `Link` shape every other list already uses.
6. **Project Detail's fetch:** adds `title` alongside `status` as a second real backend query parameter, built as its own named `titleQuery` variable, concatenated onto the same request.
7. **Project Detail's render:** renders `TitleSearch` and `SortableIssueList` together, passing the real backend-filtered `issues` into `SortableIssueList` as a prop.
8. **STRETCH (written):** reproducing Tailwind v4's own real build failure in a scratch copy of `web/` — the real error names `magic-string`, `@tailwindcss/node`, and `require() of ES Module ... not supported`, a genuine Node.js module-system mismatch under this program's pinned Node 18.20.8 + Turbopack, not a logic bug in Tailwind itself.
9. **CHALLENGE:** the real one-time `useEffect` now checks `localStorage.getItem("theme")` FIRST, falling back to the OS preference only when nothing is stored; the real per-change `useEffect` now also calls `localStorage.setItem("theme", theme)`.

Common mistakes

- Re-filtering `issues` by title inside `SortableIssueList` "just in case" — the real backend already filtered by title; duplicating it client-side risks the two filters silently drifting.
- A controlled `TitleSearch` input, or an uncontrolled `SortableIssueList` sort `<select>` — the opposite real choice from each component's own actual re-render requirement.
- Checking `localStorage` inside the wrong one of the two real `useEffect` calls in `ThemeProvider` — reads a stale value, or applies it at the wrong time.
- Describing the Tailwind v4 error as "just broken" instead of naming the real `require()` /ES-module module-system mismatch.

Exact testing instructions

BASH

```
# terminal 1
cd api && PORT=3000 npm run start

# terminal 2
cd web && npm run dev

# terminal 3
curl -s "http://localhost:3000/issues?projectId=1&status=todo&title=login"
cd web && npx tsc --noEmit && npm run build
```

Every real combination of status filter + title search + sort order produces the correct real issue list; toggling the theme and reloading the page keeps the chosen theme; `tsc` / `build` both exit clean.

What you should understand after studying

Why a real backend-owned piece of state (read from the URL) and a real client-owned piece of state (`useState`) can coexist in the same feature without either one needing to fake the other's job, and why `localStorage` must be checked in the SAME real one-time effect that used to only check the OS preference, not a separate one.